

Compiling Policy Prompts: Efficiency, Preservation, and the Cost of Reading the Request

B. Garcia

September 2026

Abstract

Large system prompts behave like policy programs: safety constraints, privacy rules, tool requirements, artifact procedures, and confirmation protocols, most of which are inactive for any one request. We ask whether a policy P and request x can be compiled into a much smaller prompt P_x such that a model answering under P_x preserves the critical obligations it satisfies under P . We report the full course of that inquiry in POLICYC, a research prototype: six deterministic compiler versions, five preregistered held-out studies (280 cases, 1,680 model executions), and five small development canaries, all on one pinned GPT-5 mini snapshot, for about \$6.9 of recorded provider spend.

Three results are established. First, efficiency is robust: every compiler cut mean input by 89.7–98.2% and uncached-equivalent cost by 55–68%, and most of the reduction on this corpus is deduplication of repeated boilerplate. Second, preservation is bounded: conditional preservation of the full prompt’s critical passes ranged from 75.8% to 86.5% against a preregistered 95% gate, and each version fixed the failure classes its development set named and lost to the classes the next fresh set named; fixes fitted to spent cases generalized worst. Third, the gate was set above the reference’s own consistency: two samples of the full prompt reproduce each other’s passes 90.4% of the time, which is an empirical baseline, not a ceiling, and every paired result must be read beside it.

The remaining evidence explains the plateau rather than patching it. Retaining the policy’s verbatim conditional text does not make the model apply the condition; quoting the request’s evidence beside the rule makes the model re-verify it; what moves behavior is a directive. A model that reads the verbatim clauses and writes directives recognized three of four fresh confirmation phrasings that six versions of deterministic reading recognized none of, matched the full prompt’s passes, and took no unsafe action where the full prompt took three. It costs about \$0.006 and 20–30 seconds per request, roughly the full prompt’s whole answer, because the full prompt is cache-served. Cheaper readers, under two contracts including one that lists the policy’s request-dependent conditions so the reader need only compare, kept the rule and lost the request; a worklist resolved by the answering model itself did not substitute. The paper’s conclusion is a diagnosis: on this provider the semantic reading that generalizes costs what a reasoning model charges for it, a sequential reader cannot reach latency parity, and the open problem is no longer selecting policy text but paying for its reading.

1 Introduction

Production system prompts are growing from short behavioral instructions into heterogeneous policy bundles. A single prompt may state when live research is mandatory, which sources are authoritative, how destructive actions must be confirmed, which personal attributes must not be inferred, what tools may be called, how files must be inspected, and which response format is appropriate. Most

user requests activate only a fraction of these rules, yet conventional inference supplies the full prompt on every request.

This creates an appealing systems question: can a request-conditioned compiler activate only the policies relevant to the current request? A successful compiler would reduce input tokens, cost, and latency while making the active contract inspectable. The failure mode is asymmetric. Dropping redundant prose is harmless; dropping one confirmation, privacy, or tool-integrity rule can be severe. Average semantic similarity is therefore an inadequate target, and the question has to be asked behaviorally: does the model, given the compiled prompt, do what it does given the whole one?

Generic prompt compression removes low-information passages or tokens while preserving downstream task accuracy [1, 2, 3, 4]. Policy compilation has a different objective: it must preserve obligations whose importance is conditional on a specific request, tool surface, artifact, or risk. Related alignment work studies explicit principles and privileged instruction levels [5, 6] but does not test request-conditioned removal of privileged policy text.

We formalize the problem as follows. Let P be a policy program, x a user request plus optional structured context, and S a selector. The compiler emits

$$P_x = \text{closure}(S(P, x)), \quad (1)$$

where closure adds every transitive policy dependency. Let $C(P, x, y)$ indicate that answer y , produced under policy condition P , satisfies all independently specified critical obligations for x . The primary target is conditional preservation:

$$\theta = \Pr[C(P_x, x, Y_{P_x}) = 1 \mid C(P, x, Y_P) = 1]. \quad (2)$$

Conditioning on full-policy success is essential: if both conditions fail, the pair says nothing about whether compilation preserved a successful obligation-bearing behavior. The primary efficiency target is

$$R_{\text{input}} = 1 - \frac{\mathbb{E}[T_{\text{input}}(P_x, x)]}{\mathbb{E}[T_{\text{input}}(P, x)]}. \quad (3)$$

This paper reports the inquiry as it ran, in three phases, and states what each phase establishes and what it does not.

1. **Deterministic compilation (compilers 0.5–0.10, held-out v1–v5).** A typed policy graph, a request-conditioned selector with dependency closure, an emitter, and successive attempts to recognize the request’s state: a surface-form predicate, a typed request state read by regexes or by a model-assisted extractor, declared branches, field contracts, and effect-aware masking. Five preregistered held-out studies gave a stable efficiency result and a stable failure to preserve.
2. **Measurement (noise floor).** The reference’s own self-consistency, computed from grades that already existed, showed the preregistered gate had been set above what a copy of the full prompt would score.
3. **Mechanism isolation (source-first arms, canaries 1–5).** Emission that keeps the policy’s original bytes, evidence bound beside rules, a model that reads the clauses and writes directives, cheaper readers, a condition index that removes the reader’s search, and a single-call worklist. Small, isolated, blind-graded canaries with kill conditions fixed before any output separated what moves behavior from what does not, and priced the one mechanism that generalized.

Section 2 places the work. Section 3 defines the measures, including the three quantities every later result reports together. Section 4 describes the apparatus. Sections 5–11 present the studies in order. Section 12 states the synthesis, Section 13 the threats to validity, and Section 14 the provenance.

2 Related Work

Prompt compression. Selective Context filters prompt content using self-information [1]. LLM-Lingua introduces budget controllers and token-level compression for efficient inference [2]; LongLLM-Lingua adapts compression to long-context retrieval and position bias [3]; LLMLingua-2 frames task-agnostic compression as token classification distilled from a larger model [4]. These methods evaluate answer quality, retrieval, or semantic fidelity. POLICYC treats policies as typed program elements and evaluates independent obligations, because infrequent policy text may be critical even when it is semantically peripheral.

Principle-guided behavior and instruction priority. Constitutional AI shows how an explicit written constitution can guide critique and preference learning [5]. The Instruction Hierarchy formalizes priority among system, developer, user, and tool-originated instructions [6]. POLICYC trains no model and introduces no hierarchy; it asks which privileged instructions can be omitted for one request without changing critical behavior.

Partial evaluation and program slicing. The compiler analogy is closer to partial evaluation than to summarization. A partial evaluator specializes a program on known inputs while preserving its semantics [7]; program slicing retains the statements that may affect a criterion. Policy prompts are not conventional programs and language models are stochastic interpreters, but the analogy motivates explicit dependencies, conservative closure, and paired semantic tests. The experiments show where it breaks: flattening a conditional policy into unconditional natural language changes behavior even when the right node was selected, and a residual program the interpreter can read is not the same as one it will execute.

Behavioral evaluation. Structural recall and behavioral preservation are distinct. A selector can retrieve every annotated node while the emitted wording weakens, strengthens, or misconstrues those rules. POLICYC never treats its own selected nodes as ground truth: each case carries independently declared obligations, and primary results come from strategy-blind answer comparison.

3 Measures

3.1 Paired preservation

For a paired sample, four determinate critical outcomes are possible: both pass; the full policy passes and the compiled prompt fails (FULLONLY); the compiled prompt passes and the full policy fails; or both fail. The estimator for Eq. 2 is

$$\hat{\theta} = \frac{n_{\text{both pass}}}{n_{\text{both pass}} + n_{\text{full only}}}. \quad (4)$$

Trial-level Wilson 95% intervals and exact two-sided McNemar tests are reported as descriptive statistics; the samples of one case are clustered and do not create independent situations. Compiler-only passes are reported and never netted against full-only losses: for policy replacement, a lost baseline success is the quantity of interest.

3.2 Self-consistency of the reference

The paired estimator conditions on one stochastic sample of the full policy passing. If a second independent sample of the full policy would not pass, some of the apparent loss is sampling variance. With n graded full-policy samples per case, every ordered pair of distinct samples (i, j) contributes to

$$\rho = \Pr[F_j \text{ passes} \mid F_i \text{ passes}], \quad (5)$$

which is Eq. 2 with the full policy in both slots: the score a byte-identical copy of the reference would obtain. ρ is a baseline, not a ceiling. A different representation can be more stable, or more correct, than the reference and exceed it; nothing in the metric forbids that.

3.3 Three quantities, reported together

From Section 7 on, every behavioral result reports, from the same samples: source-grounded correctness (pass, unsafe action, and redundant re-ask against a rubric written from the policy alone, before any output existed); preservation of the full policy’s passes, paired by sample index; and in-run self-consistency ρ . The first asks whether the arm did what the policy says; the second asks whether it did what the reference did; the third says how much the reference agrees with itself. A compiler at 94% preservation against a reference whose self-consistency is 90%, while improving source-grounded correctness, is a different result from “failed to reach 95%,” and the reports must be able to say so.

4 Apparatus

4.1 Policy representation, selection, and emission

The synthetic enterprise-agent prompt (16,191 tokens, 21 sections) is represented as 46 manually structured nodes in six YAML packs, validated at load and as a graph (no duplicate identifiers or edges, no cycles, no unknown validators). Given request x , artifact metadata, and the tool surface, the deterministic TypeScript compiler detects intents lexically, activates universal policies, matches triggers against request text and context, conservatively retains high-impact safety, privacy, and tool-integrity rules, traverses the transitive **requires** graph, and orders the result. Closure is exact with respect to declared edges; it establishes dependency completeness, not behavioral completeness. The emitter serializes the selection as compact natural-language instructions. The representation is intentionally explicit: the system compiles a structured graph and does not parse arbitrary prose into policies.

4.2 Compiler lineage

Table 1 summarizes what each version added. The 0.8 specialization stage evaluates an authored predicate after closure and swaps a node’s emitted branch when the predicate holds, failing closed otherwise. Compiler 0.9 replaced it with a compiler in the ordinary sense: a *frontend* reads the request once into a typed request state (authorization present, reported, conditional, or absent; a user-stated limit; the allowed deliverable; a refused purpose; a stated output format; which policy-required fields are stated; whether the operation is named or negated), policy nodes declare branches over that state with a declared precedence table, and a printer emits the first branch decided true. Two frontends exist: regex readers, and an *extractor*, one strict structured-output model call per request at compile time whose output is persisted, hashed, and named in the run manifest. Compiler 0.10 extended the

Table 1: Deterministic compiler lineage. Each version was frozen before its held-out set was opened; 0.10 was not frozen.

Version	Change
0.5	Compact universal kernel; selector and closure; emitter
0.6	Removed a leaked machine directive; policies from confirmed 0.5 regressions
0.7	Hardened tool availability; regressions from 0.6
0.8	Specialization stage: <code>explicit_confirmation</code> predicate with a fail-closed satisfied branch on seven confirmation nodes
0.9	Typed request state, declared branches and precedence, printer; regex and extractor frontends
0.10	Five more state facts, field contracts, exact connector scopes, effect-aware masking, extractor revision 3; unfrozen

state with five further facts, added artifact field contracts and connector scopes, and made masking effect-aware; it is a development checkpoint, never frozen (Section 6).

4.3 Source-first arms

The third phase added emission paths that keep the policy’s original bytes and make no semantic decision. `source_preserving_slice` projects the selector’s chosen nodes onto whole source sections. `source_clause_slice` projects onto a hand-audited map of 61 clauses, one per prose paragraph, line-anchored and hash-checked against the pinned source; a clause can be removed only by a trusted structural fact (an artifact type or tool declared exhaustively absent by the case), dependencies travel with it, and the twenty sections’ nine identical boilerplate lines are emitted once. `source_evidence_bare` and `_apply` add verbatim sentences of the request bound to the clauses’ declared evidence roles, with no verdict. `model_reader_slice` appends directives written by a separate model call (Section 4.4). `condition_list_slice` appends the policy’s own request-dependent sentences under one resolve-then-act instruction, with no reader (Section 11). The unchanged end-to-end compiler remains a control arm in every run.

4.4 Model reader

The reader is one structured-output call at compile time. Under *contract 1* it receives the verbatim clause slice, the request, and the declared context, and returns `{condition, finding, directive}` triples for each policy condition it finds that the request bears on. Under *contract 2* it also receives the listed conditions of the retained clauses from a frozen index (Section 11.1), and must return exactly one `{condition id, holds ∈ {yes, no, undecidable, not-applicable}, quote, directive}` per listed id; the runtime rejects a reading that does not cover the listed ids exactly once. Under both contracts only directives are rendered, after the slice; conditions, findings, verdicts, and quotes stay in the artifact for audit. An empty reading renders the bare slice, so the arm degrades to source-first retention and never manufactures a directive. Each contract is identified by a hash of its prompt, response schema, clause map, index, and input envelope; readings are persisted and named in the answer run’s manifest, so a run’s identity fixes what was read.

4.5 Runtime and provenance

A Python `asyncio` runtime executes paired trials with bounded concurrency, hard ceilings on calls, tokens, and dollars, atomic raw-response persistence, resumability without repeated paid calls, and a

Table 2: Frozen held-out studies. “Pairs” counts complete paired outputs before semantic ungradables.

Compiler	Cases	Model calls	Complete pairs	Web searches	Cost	Output cap
0.5	50	300	129/150	0	\$0.6881	2,048
0.6	50	300	139/150	41	\$1.0357	2,048
0.7	60	360	180/180	20	\$0.9333	3,072
0.8	60	360	163/180	16	\$1.0653	2,048
0.9	60	360	177/180	26	\$1.1790	3,072
Total	280	1,680	—	103	\$4.9014	—

rebuildable SQLite catalog. A keyless dry run prices every plan’s worst case before anything is sent; paid execution requires a run-specific typed confirmation. Trial identifiers bind run, case, sample, strategy, model, provider, and candidate; Python verifies compiled-prompt hashes before execution. Every run directory, reader plan, and the catalog are in the public repository (Section 14).

4.6 Evaluation protocol

Every case specifies a request, artifact context, independently authored obligations and prohibitions, critical obligation identifiers, refusal and tool expectations, and a rubric. Held-out sets v3–v5 were written by context-isolated authoring agents, each given only a brief and the synthetic prompt, cross-audited, and frozen before the corresponding compiler was opened to them; once inspected, a set became development evidence and was never reused as fresh evidence. Primary behavioral results come from strategy-blind semantic packets graded by isolated agents, with grade hashes locked before answer identifiers were joined to the private strategy map.

The canaries of Sections 9–11 tightened the protocol for small sets: an isolated author who saw only the policy and a brief chose the situations and wrote the rubric (and, in canary 5, per-case expected verdicts for the indexed conditions) before any output existed; the case file, rubric, and labels were hash-locked; kill and pass conditions were written down before the first paid call; a separate isolated grader scored one shuffled packet per trial, with tool calls and their arguments, and its transcript was audited for reads outside its workspace; grades were hash-locked before unblinding. Reader-only stages ran before answer runs, with a gate that decided which readers earned one.

5 Study 1: Five Frozen Held-Out Studies

5.1 Design

Table 2 summarizes the executions. All use `gpt-5-mini-2025-08-07`, two strategies (`full_policy` and `compiler_slice`), three samples per case, no retries, and deterministically counterbalanced dispatch. Compiler 0.5 had no provider tools; 0.6–0.9 used web and synthetic function tools. The v4 output cap was preregistered at 2,048 tokens because every v3 response had fit within it; 21 v4 executions reached the cap, and the 17 affected pairs were excluded from paired estimates and counted against coverage, never as passes. Compiler 0.9 was compiled under the extractor frontend.

Table 3: Compiled-condition changes relative to the full prompt. Positive reductions are favorable; positive latency deltas are slower.

Compiler	Input reduction	Billed-cost reduction	Uncached-cost reduction	Latency delta
0.5	98.23%	23.01%	67.80%	−19.95%
0.6	89.69%	24.50%	55.36%	+0.39%
0.7	93.75%	12.14%	59.46%	+14.63%
0.8	94.76%	18.03%	66.00%	−7.07%
0.9	92.99%	17.21%	58.69%	+13.36%

Table 4: Primary strategy-blind semantic results. Compiler 0.6 includes three ungradable pairs, so strategy pass-rate denominators differ.

Compiler	Full pass	Compiled pass	Both	Full only	Compiler only	Both fail
0.5	107/129	101/129	92	15	9	13
0.6	74/138	73/136	64	10	9	53
0.7	163/180	141/180	130	33	11	6
0.8	132/163	111/163	100	32	11	20
0.9	137/177	125/177	104	33	21	19

5.2 Efficiency

Table 3 is the most stable result in the project. Every compiler reduced mean input by at least 89.69% and uncached-equivalent cost by at least 55.36%, on five independently authored sets. Billed savings were smaller because the repeated full prompt benefited from prompt caching while compiled prompts varied by request, and because compiled prompts sometimes produced longer answers or more paid searches. Latency did not improve consistently: output length, tool activity, cache state, and reasoning all move wall-clock time, and input reduction alone does not.

5.3 Preservation

Tables 4 and 5 show that no study supports near-equivalence. Compiler 0.6 produced almost identical marginal pass rates for the two conditions, yet paired preservation was 86.49%: nine pairs passed only under the compiler while ten different pairs passed only under the full prompt. Equal averages hide swapped successes. Compiler 0.7 is a stronger negative result (79.75%, 16 distinct regressed cases, leave-one-case-out 79.38–81.25%). Compiler 0.8 on v4 is the lowest observed, 75.76%, on a set that was harder for both conditions (the full prompt itself passed 80.98% of v4 answers against 90.56% on v3). Compiler 0.9 on v5 preserved 75.91%, within a point of 0.8 on a different set, with the most compiler-only passes of any study (21), so its discordant pairs are nearly balanced and McNemar is not significant. On v5, preservation was 87.5% on cases without a tool and 42.9% on cases that required one.

All three later compilers had gates fixed before execution (Table 6). Each passed its efficiency gates and failed every behavioral one.

5.4 Where each version lost

All FULLONLY pairs of 0.7, 0.8, and 0.9 were classified after the blind grade lock (Tables 7–9). The classification is exhaustive in each case; every pair has an address in the compiler.

Table 5: Conditional preservation and uncertainty. Intervals are trial-level Wilson 95% intervals and remain descriptive because samples are clustered by case.

Compiler	Preservation	Wilson 95%	Distinct full-only cases	McNemar p
0.5	92/107 = 85.98%	78.15–91.32%	9	0.3075
0.6	64/74 = 86.49%	76.88–92.49%	6	1.0000
0.7	130/163 = 79.75%	72.93–85.21%	16	0.0013
0.8	100/132 = 75.76%	67.79–82.27%	17	0.0019
0.9	104/137 = 75.91%	68.11–82.30%	16	0.1337

Table 6: Preregistered held-out gates for compilers 0.7 (v3), 0.8 (v4), and 0.9 (v5).

Gate	Threshold	0.7	0.8	0.9
Conditional preservation	$\geq 95\%$	79.75%	Fail	75.76%
Wilson lower bound	$\geq 90\%$	72.93%	Fail	67.79%
Distinct full-only cases	≤ 3	16	Fail	17
Mean input reduction	$\geq 90\%$	93.75%	Pass	94.76%
Total billed-cost reduction	$\geq 15\%$	12.14%	Fail	18.03%
Complete paired coverage	$\geq 90\%$	100%	Pass	90.56%

Three regularities hold across the lineage. First, each version fixed the classes its development set named and lost to the classes the next fresh set named: 0.7’s largest cluster was fifteen pairs of redundant confirmation on already-confirmed actions, 0.8’s specialization stage repaired all six spent cases of that class and matched none of the four fresh phrasings an independent author wrote (“no need to check back”, “everyone’s confirmed”, “go ahead”), and 0.9’s extractor then read those phrasings correctly on every regressed case and lost the study in selection and in conditions its representation did not declare. Second, fixes fitted to spent cases generalized worst: the “I confirm” predicate of 0.8, safe on fifteen negative controls, was narrow on everything else; the blanket-approval branch of 0.9, derived from one spent case, fired on every fully specified artifact edit lacking a field contract. Third, the compiled prompt’s efficiency and its behavior were governed by different things: the prompt could be 3% of the original and still ask for a confirmation the user had just given.

6 Study 2: Compiler 0.10 and the Hidden Generality Gate

Compiler 0.10 extended the request state with current-information, deferred-work, slide-task, disclosure, and requested-reorder facts; added field contracts for spreadsheet, document, PDF, email, and calendar operations, so that a complete action requires authorization in the user’s voice, a named non-negated operation, and every operation-specific target field; added exact connector scopes; made masking effect-aware (ask-side confirmation suppresses acting connectors and keeps reads); and revised the extractor prompt. Offline, it passed its visible regression sets and a tuned five-case slice.

Before any paid study, the deterministic frontend was scored one-shot against five hidden slices authored in isolation: 180 rows, 440 assertions, labels never seen by the frontend’s author. It matched 333 of 440 assertions (75.68%) and 93 of 180 rows fully (51.67%). These are frontend label-agreement scores, not preservation, and they are not comparable to Table 5; they are also a failed gate. The slices were spent on scoring and never diagnosed. No paid run was made from 0.10, it was checkpointed rather than frozen, and the inquiry changed course: instead of another architecture, isolate the mechanisms that were losing.

Table 7: Primary attribution for compiler 0.7 full-only regressions (held-out v3).

Cause	Pairs	Cases	Dominant examples
Emitter wording / semantic loss	21	8	Redundant confirmation on already-confirmed actions, present-tense background-work fiction, forced Draft/Notes wrapper
Selector omission / misselection	7	4	Prompt-only image request treated as generation, missed image edit, forbidden spreadsheet call, “now” treated as current information
Context-interface asymmetry	2	1	Domain hint surfaced only in the compact condition and omitted from the blind packet
Primarily stochastic	3	3	Missing citation, invented legal detail, invented date

7 Study 3: The Reference’s Self-Consistency

Equation 5 was computed from grades that already existed. On held-out v5, with three blind-graded full-policy samples on 57 of 60 cases (177 ordered pairs):

$$\rho = 0.904, \quad \text{pairwise agreement } 0.851, \quad 13 \text{ of } 60 \text{ cases split}, \quad \text{full pass rate } 0.774.$$

On canary 2 (8 cases, 2 samples, a source-grounded rubric) $\rho = 1.000$; on canary 1 (4 cases) $\rho = 0.800$. A byte-identical copy of the full prompt would have scored 90.4% on v5’s preservation metric and failed the preregistered 95% gate. Compiler 0.9’s 75.91% is 14.5 points below the reference’s self-consistency, not 19 below the gate; that baseline, not the gate, is the honest comparison. The 33 lost pairs of v5 were binned by reading each pair, and the split-case list was not used to re-bin anything. From this point every paired result in the project reports in-run ρ beside it.

8 Study 4: Source-First Emission, Offline

The 0.10 failure analysis had argued that transformation loss (authored wording, branch resolution, masking) and selection loss should be separated, which requires an arm that keeps the source’s own words. Two offline measurements on the 97 visible development cases established what such an arm can and cannot do.

Retention is mostly deduplication. Whole-section retention of the selector’s chosen nodes leaves a 46.59% mean token reduction. The clause map leaves one prompt of 3,259 tokens, identical for every visible case, 79.87% below the full policy. Nothing was pruned in any case: no visible case declares its context exhaustive, so no structural fact establishes irrelevance under the rules adopted, and every clause is retained. The entire reduction is the audited deduplication of boilerplate: of the source’s 16,123 section tokens, 13,371 are nine bullet lines repeated in twenty sections and 2,449 are substantive prose. A real policy that does not repeat itself would gain almost nothing from this approach; the 90%+ reductions of Table 3 were partly this corpus’s redundancy. With an exhaustive declaration, pruning tops out around 10 of 61 clauses.

Authoring is where conditions die. Tracing five cases through the compiler stage by stage found that in every one at least one authored YAML default had turned a conditional source sentence into an unconditional instruction before any branch ran: confirmation “when appropriate” became always

Table 8: Primary attribution for compiler 0.8 full-only regressions (held-out v4).

Cause	Pairs	Cases	Dominant examples
Emitter: tool availability emitted, user’s limit not	11	6	“just tell me what to watch for”, “only describe”: the compiled prompt listed the tool and the domain policy, and the model called it
Emitter: confirmation state, predicate unmatched	7	4	“no need to loop back to me”, “everyone’s confirmed and I don’t need another readback”, “already cleared with everyone”, “go ahead”
Emitter: confirmation asked without target/scope text	7	3	“clear out” read as delete; publish request answered by inventing the content
Emitter: privacy-minimization text not emitted	2	1	Availability readout replaced by questions naming the private appointment
Selector: web policy activated against a stated no-lookup	3	1	Definitional accounting question with “don’t look it up”
Primarily stochastic	2	2	Simulated-inspection claim; missed deadline in a readback

ask; “notes only when they help” became a mandatory format; “inspect before editing” became a mandatory tool call; “prefer drafts” became “do not draft”. Only 18 of 97 visible cases take a branch at all; 79 ship those defaults.

9 Study 5: Canaries 1–3, from Retention to a Reader

Three small paid canaries then tested, in order, whether the answering model applies a preserved condition unaided, whether quoting the request’s evidence beside the rule helps, and whether a model that reads the clauses and writes directives does what the deterministic reader could not. Each used fresh cases from an isolated author, a rubric locked before outputs, and kill conditions fixed in advance; canaries 2 and 3 used an isolated blind grader. Cases from every canary are spent.

9.1 Canary 1: a preserved condition is not an applied one

Four cases, three arms (full, clause slice, current compiler), two samples, \$0.06. Passes of 8: full 5, clause 4, compiler 6. On the one case that tests the hypothesis, a send the user had reviewed word for word (“I have read the final wording below and it is exactly what I want ... Send it now”), the clause slice re-asked in both samples with the verbatim confirmation rule in its prompt, and so did the compiler, whose regex frontend read `authorization: absent`. The full policy sent in both samples. The only arm that acted on turn one anywhere in the canary was the compiler, on a case where it emitted a directive; the same reader that produced that directive forwarded privileged material on a misread. Edge and danger were one mechanism.

Table 9: Primary attribution for compiler 0.9 full-only regressions (held-out v5), with the extractor’s recorded state.

Bin	Pairs	Cases	Dominant examples
Selector gap	15	6	“the ecb deposit facility rate as it stands today”, “the mileage rate thats in effect right now”: current information the trigger vocabulary does not mark, so the mandated web node was never selected and a correctly read “don’t go online” stood; “deploy” selecting confirmation on a log diagnosis
Undeclared condition	9	4	Fully specified spreadsheet and slide edits with no field contract for the artifact, so the action was undecidable and the blanket-approval branch asked; counsel’s thread forwarded to an outside contractor on a correct read of “i cleared it with legal already”, the one unsafe class
Frontend misread	7	4	Limits read where none was stated (“i mainly care about revenue and churn”); one stated format not read
Stochastic	2	2	One sample each on prompts that carried the obligation

Table 10: Canary 3, 16 trials per arm. Kept: full-policy passes reproduced, paired by sample; the full policy’s own other sample kept 2 of 6.

Arm	Pass	Unsafe action	Redundant re-ask	Kept of 6 full passes
<code>compiler_slice</code>	2	0	8	1/6
<code>source_clause_slice</code>	5	2	3	2/6
<code>full_policy</code>	6	3	2	2/6 (vs. itself)
<code>model_reader_slice</code>	6	0	2	4/6

9.2 Canary 2: evidence beside the rule makes it worse

Eight cases, five arms (adding the two evidence-binding frames), two samples, \$0.16. Passes of 16: full 6, clause 5, evidence-bare 5, evidence-apply 5, compiler 4. On satisfied cases the evidence arms re-asked on 4 of 6 samples against the full policy’s 2 of 6, and re-asked *about the items the evidence block quoted*: the user’s statement that attendees had confirmed became a checklist for the user to confirm again. A confirmed archive was re-asked 10 of 10 times across all arms; no policy-text prompt archived on turn one. The apply frame produced one soft unsafe action. Both kill conditions were met and evidence binding was abandoned. The lesson, carried into every later design: the model re-verifies quoted facts and follows instructions; what moves behavior is a directive.

9.3 Canary 3: a model reads what the regex could not

Eight cases, four arms, two samples, 64 graded answers and 8 reader calls; \$0.41 including two superseded reader runs. The reader (`gpt-5-mini`, default reasoning, contract 1) saw the clause slice (about 3,600 tokens), the request, and the declared context; its prompt was written before any case existed and never revised.

On the four fresh confirmation phrasings (“Word for word the text is...”, “read it back and it is exactly...”, “Carry on...”, and a reviewed line in a two-part request), the deterministic frontend read `authorization: absent` on every one, so the compiler fell to its default ask on every sample: 0 of 6 confirmed trials acted. The model reader read three of the four as confirmed, wrote “proceed ... do not ask for further confirmation”, and the answering model acted on all six corresponding

Table 11: Canary 3 economics, mean per answer. The reader call: 3,632 input tokens, 2,901 output (2,016 reasoning), \$0.00592, 24 s median.

Arm	Prompt tokens	Answer cost	Answer latency	Pipeline cost
full_policy	16,191	\$0.00462	8.0 s	\$0.00462
compiler_slice	431	\$0.00422	11.0 s	\$0.00422
source_clause_slice	2,929	\$0.00293	8.5 s	\$0.00293
model_reader_slice	3,294	\$0.00353	7.6 s	\$0.00945

trials. Its one miss was in the safe direction (a fully specified reschedule resolved as an obligation to ask, as the full policy itself did on one sample). The reader arm was the only acting arm with zero unsafe actions: the full policy forwarded privileged counsel material once and overwrote spreadsheet formulas twice, the clause slice overwrote them twice, and the reader’s directive to “inspect the range for formulas first” left the range alone.

Table 11 is the catch. The reader call costs more than the whole full-policy answer, because 71% of the full prompt’s 17k input is cache-served at a tenth of fresh price while the reader spends two thousand reasoning tokens; the pipeline is twice the cost and four times the latency of doing nothing, with a prompt 80% smaller. The compiler’s 431-token prompt is *more* expensive than the full policy because it emits 1,330 output tokens of re-asking.

Canary 3 reframed the diagnosis. The story had been that compression destroyed a quarter of the policy’s semantics; the evidence reads that compression succeeded and six versions were spent reconstructing natural-language policy state with the wrong reader. What generalized to fresh phrasing was not an ontology or a longer regex but a model reading the source clauses, because the ways a person says “I approve” are not enumerable and every enumeration was tested on the next set’s phrasings outside it. The architecture to take seriously became

$$(P_x, x) \xrightarrow{\text{semantic reader}} D_x, \quad (P_x + D_x, x) \xrightarrow{\text{answer model}} y, \quad (6)$$

and the bottleneck moved to the cost of the reader.

10 Study 6: Cheaper Readers under Contract 1

The break-even budget for the reader follows from Table 11: the reader arm’s answer already runs at \$0.0035 and 7.6 s against the full policy’s \$0.0046 and 8.0 s, so the reader may spend about \$0.001 and well under a second for the pipeline to beat the full policy on both. Canary 4 held the reader contract fixed and varied only the model and reasoning effort, on eight fresh cases whose rubric declared, per case, whether the request supplies the policy’s confirmation. A confirmation-ledger gate was fixed before the first call: a reader whose directive on a confirmed action tells the assistant to obtain confirmation, on more than one of the five confirmed actions, gets no answer run.

All three were killed at \$0.026 total. Minimal effort removes reasoning tokens entirely, and with it the reading: mini at minimal wrote seven contradictory resolutions on one confirmed forward and settled on “only after the user confirms”; nano at minimal returned `{"resolutions": []}` on every confirmed action; nano at low read every confirmed action as “ask” and the one privileged external disclosure as “This condition holds. Proceed to send the privileged memo”, the misread of canary 1 again. Every cheap reader’s directives on the unconfirmed cases were correct. What the cheap readers kept was the rule; what they lost was the request, which is the failure the deterministic frontend had.

Table 12: Canary 4 readers, contract 1, eight cases. “Proceed” counts confirmed actions on which the reader’s directive said to act without re-asking.

Reader	Cost/case	Latency	Output (reas.)	Proceed/5	Outcome
mini, default (canary 3)	\$0.0059	24 s	2,901 (2,016)	3/4	reference reader
mini, minimal	\$0.0026	11.2 s	1,113 (0)	2	killed; contradicts itself within a case
nano, minimal	\$0.00014	1.1 s	128 (0)	0	empty readings on 6 of 8 cases
nano, low	\$0.00047	5.3 s	1,021 (816)	0	killed; “proceed” on the privileged external send

11 Study 7: Condition-Indexed Reading and a Single-Call Worklist

11.1 Removing the search

Canary 4’s readers appeared to fail at a search: asked to find the request-dependent rules in the slice, they emitted the salient rule instead of comparing the request to it. Contract 2 does the search once, offline. A source-first *condition index* was audited over the entire policy under one criterion: a condition is a sentence whose directive for this turn depends on what the request itself states (whether the user has confirmed, authorized, requested, or limited something, or left something unstated). Every one of the 61 clauses was classified; triggers (which name when a duty applies), definitions, conditions on facts the request cannot settle (tool failure, artifact contents), compiler meta-instructions, and restatements were excluded. Sixteen conditions over twelve clauses survived, each a verbatim sentence and the verbatim span the request must meet, tested as substrings of the pinned source. Six of the sixteen belong to classes no canary had exercised; the index was frozen, and the owner forbade adding, removing, paraphrasing, or tuning any entry on subsequent evidence.

The retained clauses’ indexed conditions are listed for the reader, and the reader answers each exactly once (Section 4.4). Because the clause map retains conservatively, a request lists 12–14 conditions, of which about eight are inert for it; that cost was measured rather than trimmed. Alongside, `condition_list_slice` appends the same sentences to the answer prompt under one instruction to resolve them from the request’s words before acting, so that the two arms separate the question “is a finite worklist enough?” from “does a separate reader matter?”

11.2 Reader stage

Eight fresh cases; the isolated author also wrote, per case, the expected verdict for every applicable indexed condition, so the gate became mechanical: each reader’s `holds` against the locked labels. Kill: more than one disagreement on the confirmation conditions of confirmed actions, or any “yes” on the unconfirmed privileged disclosure or on the unexamined spreadsheet range.

The reference reader reads: every confirmed action’s specific condition right, both unconfirmed disclosures right, 57 of 71 inert conditions answered not-applicable. Its errors are the policy’s own overlaps (a generic confirmation sentence read “no” beside a calendar-specific one read “yes”) and one hazard reading (“swap whatever is in those six cells” taken as an explicit request to overwrite), which the locked rule counted as the unsafe direction. It also got dearer: listing thirteen conditions made the only reader that reads think for 2,680 tokens. The cheap readers still did not read, and the contract made the failure countable rather than a judgment: mini at minimal filled the schema with “yes” and emitted the rule in every directive (thirteen directives contain the literal JSON field names); nano at

Table 13: Canary 5 readers, contract 2, eight cases, 33 labeled conditions. “Consistent” counts confirmation conditions whose directive matches the verdict (a “yes” carries a proceed directive, a “no” an ask).

Reader	Cost/case	Latency	Output (reas.)	Valid	Agree	Consistent; outcome
mini, default	\$0.0077	30.6 s	3,578 (2,680)	8/8	25/33	11 of 12; killed on one “proceed” for the unexamined range
mini, minimal	\$0.0024	10.0 s	958 (0)	8/8	17/33	1 of 11; “yes” on all eight confirmations, ask-directive on every one
nano, minimal	\$0.0003	3.4 s	605 (0)	8/8	3/33	noise
nano, low	\$0.0009	8.9 s	2,176 (1,776)	5/8	8/33	three readings did not cover the listed ids

Table 14: Canary 5 answer run, 16 trials per arm. In-run $\rho = 4/5$, one case split. No arm passed where the full policy failed.

Arm	Prompt tokens	Pass	Unsafe	Re-ask	Kept of 5	Cost/answer
full_policy	16,402	5	2	0	—	\$0.00297
compiler_slice	690	4	0	8	4/5	\$0.00274
source_clause_slice	3,162	4	0	7	4/5	\$0.00195
condition_list_slice	3,533	3	0	4	3/5	\$0.00199

low could not follow the list. Removing the search did not restore the comparison.

11.3 Answer run

No reader earned an answer run under the locked rule. The owner ran the four arms without a reader to answer the worklist question: **full_policy**, **compiler_slice**, **source_clause_slice**, **condition_list_slice**, two samples, 64 calls, \$0.154, blind-graded from one packet per trial with tool calls and arguments.

A harness effect must be stated first. The runtime records function calls and does not execute them, and the author’s tool schemas exposed identifier parameters; on three of the five confirmed cases every arm’s only call was a search, lookup, or list followed by an empty turn, the full policy included. Those six pairs fail for everyone and carry no information about any arm; the confirmed-action mechanism was observable on two cases instead of five.

On the confirmed archive (“Go ahead and do it”, scope fully bounded) the condition list re-asked on both samples, with the sentence “Confirmation should specify the target, scope, operation, and consequence” in its prompt under an instruction to resolve it first; the full policy archived once and searched once. The condition list cut redundant re-asks against the bare slice (4 versus 7), was the only slice to name the formula hazard before writing, and kept fewer of the full policy’s passes than the bare slice. The full policy was again the only arm to take an unsafe action, overwriting the unexamined range on both samples. A resolution the answering model performs for itself does not behave like a directive it is given. Latency was equal across the slices and the full policy (7.6–7.8 s); the compiled arm cost a third less per answer than the full policy with the same or fewer passes.

12 Synthesis

12.1 What the evidence establishes

Efficiency is robust and partly an artifact of redundancy. Five frozen studies and every canary produced one to two orders of magnitude less input and at least 55% lower uncached-equivalent cost. On this corpus, 83% of the source’s tokens are repeated boilerplate, and an audited deduplication alone reaches 80%; genuine per-request pruning under conservative rules reaches about 10 of 61 clauses. Prompt caching then erodes the bill: a full prompt served 71–78% from cache is cheap per call, and the compiled arm’s saving was a third, not an order of magnitude.

Preservation is bounded, and the bound has been explained rather than patched. Deterministic compilation preserved 75.8–86.5% of the reference’s critical passes on five fresh sets against a reference that reproduces itself 90.4% of the time. The losses had addresses in every version, the addresses moved with each fresh set, and the fixes fitted to spent cases generalized worst. Three canaries then located the mechanism: the model acts on directives, not on preserved conditions or quoted evidence; and the directive that generalizes to fresh phrasing comes from a model reading the source clauses, not from any enumeration of how people say “I approve.”

The semantic reading that generalizes costs what a reasoning model charges for it. The one reader that read fresh confirmations correctly and took no unsafe action spends \$0.006–0.008 and 20–30 seconds per request; on this provider that is the whole full-policy answer, and under a contract that lists the conditions it costs more, not less. Three cheaper configurations under two contracts kept the rule and lost the request, and one of them read the privileged disclosure as authorized. A sequential reader cannot reach latency parity in this design: the reader arm’s own answer leaves under half a second for the read.

The reference is not an oracle. The full prompt took three unsafe actions in canary 3 and two in canary 5, and re-asks a third of the time on confirmed actions in canary 2. Preservation measures agreement with a reference that is itself only 90% self-consistent and sometimes wrong in the dangerous direction; source-grounded correctness must be reported beside it, and an arm can legitimately be safer than the reference while preserving less of it.

12.2 What it does not establish

No preservation rate for the reader architecture: canary 3 is eight cases with an in-run ρ of 2 of 6, and the direction is consistent with the hypothesis while the magnitude is unmeasured. Whether the reference reader’s directives under contract 2 change downstream behavior: the answer run with its readings was designed and not run. Anything about other providers or model families, where cached-input pricing or reasoning cost may invert the economics either way. Anything about real organizational prompts, which do not repeat themselves twenty times.

12.3 The economics, stated

Three quantities describe the bill and must be kept apart: actual billed cost, generation-only cost, and uncached-equivalent cost. The first is what the user pays and is dominated by caching; the last isolates the compiler’s context effect. A compiled prompt’s saving is bounded by the cache discount on

the prompt it replaces, and any per-request semantic step is charged at fresh-output prices. On this provider the full policy’s answer costs about \$0.003–0.005; a reader that fits under it must produce its reading in a few hundred output tokens with no reasoning, and no such reader read.

13 Threats to Validity

Structured input. The compiler begins from manually structured YAML nodes and a hand-audited clause map and condition index. Automatic extraction of policies, dependencies, and conditions from arbitrary prose remains unsolved.

Single model and synthetic environment. All paid studies use one pinned GPT-5 mini snapshot, its nano sibling as a reader candidate, and a synthetic enterprise policy whose redundancy inflates compression. Results may differ across models, providers, and real prompts.

Changing held-out sets and small canaries. Each compiler was evaluated on a newly authored set, which protects holdout discipline and prevents controlled cross-version comparison. The canaries have four to eight cases and two samples; they separate mechanisms and price them, and they do not estimate rates.

Clustered samples. Three (or two) generations per case measure model variability, not independent situations. Intervals and tests are descriptive.

Agents as authors and graders. Cases, rubrics, labels, and grades were produced by isolated language-model agents, not human experts; isolation was audited from transcripts. The canary 3 reader prompt and the canary 3 situations were chosen by the same agent that ran the study; canaries 4 and 5 removed that link by letting the author choose situations within condition classes. Human inter-annotator reliability is not claimed.

Harness. Tool calls are recorded, not executed, so a turn whose first call is a lookup ends; canary 5 lost three confirmed cases to identifier parameters that invited a lookup. The context contract was asymmetric in one v3 pair. Both are documented and neither was corrected after unblinding.

Execution amendments. Provider-adapter and accounting fixes occurred during v0.6 and v0.7 execution and an input-reservation fix before v4; they were narrowly scoped, documented before unblinding, and did not touch cases, prompts, obligations, or parameters.

Spent evidence. Every held-out set, hidden slice, and canary set is spent; the compiler-0.10 five-case “held-back” slice was exposed during development and is recorded as tuned. Nothing in this paper reuses a spent set as fresh evidence.

14 Reproducibility

The implementation, every run directory (manifests, artifacts, trials, raw provider responses, blind packets, answer maps, reader plans and readings, budgets, reports), the catalog (29 runs, 2,478 trials),

Table 15: Primary provenance. Held-out commits identify the frozen source that constructed each run; canary identities are the run and plan ids in the repository.

Study	Identity	Notes
Held-out v1 (0.5)	run_f5f8e50b9931f65f	commit 53fc3c01f77b64910d7fa5777eda502c383f5bc1
Held-out v2 (0.6)	run_f3c3cdb0e2d1e368	commit 0e2db600496aa3489da582f88a63f4da03b998ca
Held-out v3 (0.7)	run_c019d419734c6c5e	commit d81862808f66b5fb80b5c3bcb2f06764a8a23f03
Held-out v4 (0.8)	run_16f13bd723767d59	commit f8c74cdc28157af2c4daaf42b328c4724e032e11
Held-out v5 (0.9)	run_0129a7e9a2e6730b	commit 63b66ec7c0ad1774efe8441dcc7aa7698848381c; extractor plan ext_63d9fbf0c9ea70f9
Noise floor	eval/audits/noise-floor.md	computed from the v5 blind grades; no calls
Canary 1	run_1075116c272f648f, top-up run_6e399eff52fb065a	\$0.0616
Canary 2	run_21f44ffbbc59a42e	\$0.1639
Canary 3	reader plan read_0295ee2fa56bccfc; run_24276f1d717b91cd, top-up run_8f9985f32b27fc15	\$0.411 incl. two superseded reader runs; contract 1 800e121af76e...
Canary 4	read_7887a148a87811aa, read_628f58d3ab41b3bf, read_13122ceb124e3403	\$0.026; no answer run
Condition index	09a26030a84c6db7...	eval/audits/condition-index.md; contract 2 f97e195a687b...
Canary 5	read_97da4f1c2854b8c4, read_cead37ca159cd981, read_0ad643f71f43dfe6, read_4e73044c98fd6639; run_f809810fe2c2a42a	\$0.245; grades ac9901c0... locked before unblinding

the audits, locks, rubrics, labels, and grades are in the public repository at <https://github.com/bgar324/policyc>. Table 15 records the primary identities.

Every held-out and canary set, rubric, label file, and grade file is hash-locked in a per-study lock under `output/source-slice-offline/` or in the corresponding audit; each lock records the authorization form of every paid call, the catalog counts before and after, and the secret-scan result. Each canary report lists the keyless dry-run commands that rebuild its plans without a key.

15 Conclusion

POLICYC set out to compile a policy prompt into a request-specific slice that preserves the model’s critical behavior. It established that most policy text is inactive for most requests and that a compiled prompt reliably removes it, and that on a redundant corpus much of the removal is deduplication. It established, on five preregistered sets, that deterministic compilation preserves about three quarters to six sevenths of the reference’s critical passes and that the reference reproduces itself nine times in ten, so that the remaining gap is real but smaller than the gate implied. It then established what the gap is made of: not selection, and not compression, but reading. The model acts on directives; the

directive that generalizes to how people actually write comes from a model reading the policy’s own sentences; that reading costs, on this provider, about what the whole full-policy answer costs, and every attempt to buy it cheaper, with less reasoning, a smaller model, or a contract that removed the search, produced a reader that kept the rule and lost the request. A worklist the answering model resolves for itself did not substitute.

The research outcome is a bound with a mechanism attached. Request-conditioned compilation of policy prompts is limited by the price of reading the request, not by the difficulty of slicing the policy; and reading the request is a task the cheap tiers of this provider do not perform under either contract tried. Whether that price falls with better models, better caching of readings across repeated requests, or a fast deterministic path with semantic fallback on the ambiguous fraction of traffic is the question this work leaves open, with the instruments to answer it: frozen sets, locked rubrics, a condition index over the source, a contract that makes a reading a checkable verdict, and a gate that decides before money is spent.

References

- [1] Y. Li. Unlocking Context Constraints of LLMs: Enhancing Context Efficiency of LLMs with Self-Information-Based Content Filtering. *arXiv:2304.12102*, 2023. <https://arxiv.org/abs/2304.12102>.
- [2] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. *arXiv:2310.05736*, 2023. <https://arxiv.org/abs/2310.05736>.
- [3] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. *arXiv:2310.06839*, 2023. <https://arxiv.org/abs/2310.06839>.
- [4] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang, Q. Lin, V. Rühle, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and D. Zhang. LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. *arXiv:2403.12968*, 2024. <https://arxiv.org/abs/2403.12968>.
- [5] Y. Bai et al. Constitutional AI: Harmlessness from AI Feedback. *arXiv:2212.08073*, 2022. <https://arxiv.org/abs/2212.08073>.
- [6] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. *arXiv:2404.13208*, 2024. <https://arxiv.org/abs/2404.13208>.
- [7] N. D. Jones, C. K. Gomard, and P. Sestoft. *Partial Evaluation and Automatic Program Generation*. Prentice Hall, 1993.